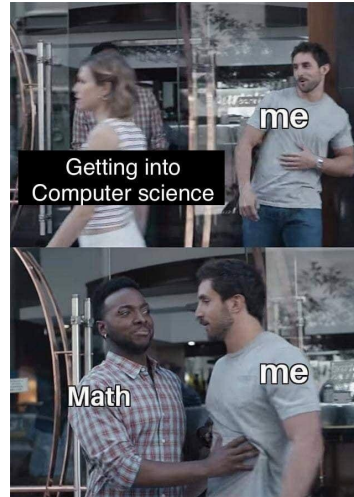


D1: Asymptotic Notations and Proof Methods Review



Sec 16: Fr 12:30pm-1:30pm FXB 1024
IA: Nick Frieler

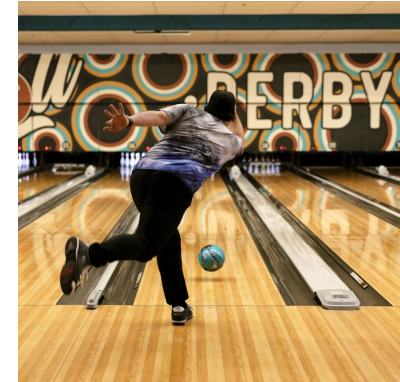
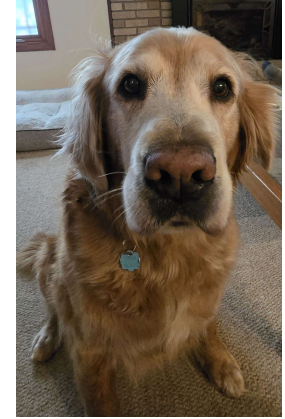
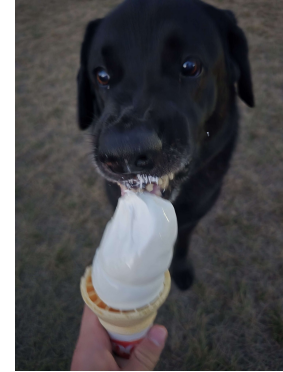
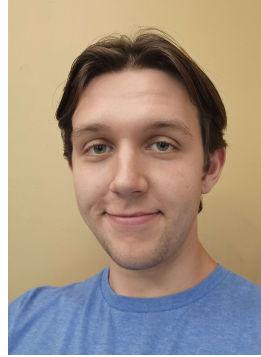
Agenda

- ▶ Introduction
- ▶ Asymptotic notation
- ▶ Proof methods review
- ▶ Some worksheet problems

Introduction

Who am I?

- ▶ Nick Frieler
- ▶ Senior in CS, Math, and Physics
- ▶ 2nd semester w/ 376
 - ▶ 1st semester leading a discussion
- ▶ Hobbies
 - ▶ Rifle shooting, walking/hiking with my dogs, bowling
- ▶ Office hours
 - ▶ Tuesday 1-2:30, East Hall Math Atrium
 - ▶ Wednesday 12:30-3:30, OH Zoom



About Discussion Attendance

- ▶ You may attend any section (try to be consistent if possible)
- ▶ If you attend at least 10 of 14 discussions/workshops throughout the semester, then you get 3% for free for your overall grade and your exam scores weigh 3% less

Assignment	Weight
Homeworks	40%
Exams (midterm and final)	56-60%
Discussion/workshop attendance	0-3%
Course evaluations	0-1%
Total	100%

Proof Methods Review

Direct proof ($P \Rightarrow Q$)

Direct proof ($P \Rightarrow Q$)

- ▶ “If P , then Q ”, also called an *implication*

Direct proof ($P \Rightarrow Q$)

- ▶ “If P , then Q ”, also called an *implication*
- ▶ We assume that statement P is **true**, and then use valid mathematical deduction to *directly* conclude that Q must also be **true**

Direct proof ($P \Rightarrow Q$)

- ▶ “If P , then Q ”, also called an *implication*
- ▶ We assume that statement P is **true**, and then use valid mathematical deduction to *directly* conclude that Q must also be **true**
- ▶ Ex: let’s prove the implication “ n is odd $\Rightarrow n^2$ is odd”

Direct proof ($P \Rightarrow Q$)

- ▶ “If P , then Q ”, also called an *implication*
- ▶ We assume that statement P is **true**, and then use valid mathematical deduction to *directly* conclude that Q must also be **true**
- ▶ Ex: let’s prove the implication “ n is odd $\Rightarrow n^2$ is odd”
 - ▶ We begin by assuming n is odd

Direct proof ($P \Rightarrow Q$)

- ▶ “If P , then Q ”, also called an *implication*
- ▶ We assume that statement P is **true**, and then use valid mathematical deduction to *directly* conclude that Q must also be **true**
- ▶ Ex: let’s prove the implication “ n is odd $\Rightarrow n^2$ is odd”
 - ▶ We begin by assuming n is odd
 - ▶ $\Rightarrow n = 2m + 1$ for some integer m

Direct proof ($P \Rightarrow Q$)

- ▶ “If P , then Q ”, also called an *implication*
- ▶ We assume that statement P is **true**, and then use valid mathematical deduction to *directly* conclude that Q must also be **true**
- ▶ Ex: let’s prove the implication “ n is odd $\Rightarrow n^2$ is odd”
 - ▶ We begin by assuming n is odd
 - ▶ $\Rightarrow n = 2m + 1$ for some integer m
 - ▶ $\Rightarrow n^2 = (2m + 1)^2 = 4m^2 + 4m + 1 = 2(2m^2 + 2m) + 1$

Direct proof ($P \Rightarrow Q$)

- ▶ “If P , then Q ”, also called an *implication*
- ▶ We assume that statement P is **true**, and then use valid mathematical deduction to *directly* conclude that Q must also be **true**
- ▶ Ex: let’s prove the implication “ n is odd $\Rightarrow n^2$ is odd”
 - ▶ We begin by assuming n is odd
 - ▶ $\Rightarrow n = 2m + 1$ for some integer m
 - ▶ $\Rightarrow n^2 = (2m + 1)^2 = 4m^2 + 4m + 1 = 2(2m^2 + 2m) + 1$
 - ▶ $\Rightarrow n^2$ is odd

Proof by contrapositive ($\neg Q \Rightarrow \neg P$)

Proof by contrapositive ($\neg Q \Rightarrow \neg P$)

- ▶ “If not Q, then not P”, this is *equivalent to* $P \Rightarrow Q$

Proof by contrapositive ($\neg Q \Rightarrow \neg P$)

- ▶ “If not Q, then not P”, this is *equivalent to* $P \Rightarrow Q$
- ▶ We assume that statement Q is **false**, and then use valid mathematical deduction to *directly* conclude that P must be **false**

Proof by contrapositive ($\neg Q \Rightarrow \neg P$)

- ▶ “If not Q, then not P”, this is *equivalent to* $P \Rightarrow Q$
- ▶ We assume that statement Q is **false**, and then use valid mathematical deduction to *directly* conclude that P must be **false**
 - ▶ This *indirectly* proves that $P \Rightarrow Q$

Proof by contrapositive ($\neg Q \Rightarrow \neg P$)

- ▶ “If not Q, then not P”, this is *equivalent to* $P \Rightarrow Q$
- ▶ We assume that statement Q is **false**, and then use valid mathematical deduction to *directly* conclude that P must be **false**
 - ▶ This *indirectly* proves that $P \Rightarrow Q$
- ▶ Ex: let’s prove “ n^2 is even $\Rightarrow n$ is even” using the contrapositive

Proof by contrapositive ($\neg Q \Rightarrow \neg P$)

- ▶ “If not Q, then not P”, this is *equivalent to* $P \Rightarrow Q$
- ▶ We assume that statement Q is **false**, and then use valid mathematical deduction to *directly* conclude that P must be **false**
 - ▶ This *indirectly* proves that $P \Rightarrow Q$
- ▶ Ex: let’s prove “ n^2 is even $\Rightarrow n$ is even” using the contrapositive
 - ▶ The contrapositive is “ n is odd $\Rightarrow n^2$ is odd”

Proof by contrapositive ($\neg Q \Rightarrow \neg P$)

- ▶ “If not Q, then not P”, this is *equivalent to* $P \Rightarrow Q$
- ▶ We assume that statement Q is **false**, and then use valid mathematical deduction to *directly* conclude that P must be **false**
 - ▶ This *indirectly* proves that $P \Rightarrow Q$
- ▶ Ex: let’s prove “ n^2 is even $\Rightarrow n$ is even” using the contrapositive
 - ▶ The contrapositive is “ n is odd $\Rightarrow n^2$ is odd”
 - ▶ We already know this is true!

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction
 - ▶ Therefore, it does not make sense to have assumed P was **false** in the first place

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction
 - ▶ Therefore, it does not make sense to have assumed P was **false** in the first place
- ▶ Ex: let's prove “all primes greater than 2 are odd” using contradiction

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction
 - ▶ Therefore, it does not make sense to have assumed P was **false** in the first place
- ▶ Ex: let's prove “all primes greater than 2 are odd” using contradiction
 - ▶ We begin by assuming that the above statement is **false**

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction
 - ▶ Therefore, it does not make sense to have assumed P was **false** in the first place
- ▶ Ex: let's prove “all primes greater than 2 are odd” using contradiction
 - ▶ We begin by assuming that the above statement is **false**
 - ▶ “At least one prime greater than 2 is even”, let p be such a prime

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction
 - ▶ Therefore, it does not make sense to have assumed P was **false** in the first place
- ▶ Ex: let's prove “all primes greater than 2 are odd” using contradiction
 - ▶ We begin by assuming that the above statement is **false**
 - ▶ “At least one prime greater than 2 is even”, let p be such a prime
 - ▶ $\Rightarrow p = 2k$ for some integer k

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction
 - ▶ Therefore, it does not make sense to have assumed P was **false** in the first place
- ▶ Ex: let's prove “all primes greater than 2 are odd” using contradiction
 - ▶ We begin by assuming that the above statement is **false**
 - ▶ “At least one prime greater than 2 is even”, let p be such a prime
 - ▶ $\Rightarrow p = 2k$ for some integer k
 - ▶ $\Rightarrow 2k > 2$

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction
 - ▶ Therefore, it does not make sense to have assumed P was **false** in the first place
- ▶ Ex: let's prove “all primes greater than 2 are odd” using contradiction
 - ▶ We begin by assuming that the above statement is **false**
 - ▶ “At least one prime greater than 2 is even”, let p be such a prime
 - ▶ $\Rightarrow p = 2k$ for some integer k
 - ▶ $\Rightarrow 2k > 2$
 - ▶ $\Rightarrow k > 1$

Proof by contradiction ($\neg P \Rightarrow \text{false}$)

- ▶ We assume that statement P is **false**, and then use valid mathematical deduction to derive a logical contradiction
 - ▶ Therefore, it does not make sense to have assumed P was **false** in the first place
- ▶ Ex: let's prove “all primes greater than 2 are odd” using contradiction
 - ▶ We begin by assuming that the above statement is **false**
 - ▶ “At least one prime greater than 2 is even”, let p be such a prime
 - ▶ $\Rightarrow p = 2k$ for some integer k
 - ▶ $\Rightarrow 2k > 2$
 - ▶ $\Rightarrow k > 1$
 - ▶ $\Rightarrow$ We can write a prime p as the product of two numbers both greater than 1 $\Rightarrow \Leftarrow$

Induction

What is induction?

What is induction?

- ▶ Induction is a convenient way to prove many (countably ∞) related mathematical statements at once

What is induction?

- ▶ Induction is a convenient way to prove many (countably ∞) related mathematical statements at once
- ▶ What do we mean by “related mathematical statements”?

What is induction?

- ▶ Induction is a convenient way to prove many (countably ∞) related mathematical statements at once
- ▶ What do we mean by “related mathematical statements”?
- ▶ Ex: let $P(n)$ be the statement “ $n^2 = 1 + 3 + 5 + \dots + (2n+1)$ ”

What is induction?

- ▶ Induction is a convenient way to prove many (countably ∞) related mathematical statements at once
- ▶ What do we mean by “related mathematical statements”?
- ▶ Ex: let $P(n)$ be the statement “ $n^2 = 1 + 3 + 5 + \dots + (2n+1)$ ”
 - ▶ $P(1)$ is then “ $1^2 = 1$ ”

What is induction?

- ▶ Induction is a convenient way to prove many (countably ∞) related mathematical statements at once
- ▶ What do we mean by “related mathematical statements”?
- ▶ Ex: let $P(n)$ be the statement “ $n^2 = 1 + 3 + 5 + \dots + (2n+1)$ ”
 - ▶ $P(1)$ is then “ $1^2 = 1$ ”
 - ▶ $P(2)$ is then “ $2^2 = 1 + 3$ ”

What is induction?

- ▶ Induction is a convenient way to prove many (countably ∞) related mathematical statements at once
- ▶ What do we mean by “related mathematical statements”?
- ▶ Ex: let $P(n)$ be the statement “ $n^2 = 1 + 3 + 5 + \dots + (2n+1)$ ”
 - ▶ $P(1)$ is then “ $1^2 = 1$ ”
 - ▶ $P(2)$ is then “ $2^2 = 1 + 3$ ”
 - ▶ $P(3)$ is then “ $3^2 = 1 + 3 + 5$ ”

What is induction?

- ▶ Induction is a convenient way to prove many (countably ∞) related mathematical statements at once
- ▶ What do we mean by “related mathematical statements”?
- ▶ Ex: let $P(n)$ be the statement “ $n^2 = 1 + 3 + 5 + \dots + (2n+1)$ ”
 - ▶ $P(1)$ is then “ $1^2 = 1$ ”
 - ▶ $P(2)$ is then “ $2^2 = 1 + 3$ ”
 - ▶ $P(3)$ is then “ $3^2 = 1 + 3 + 5$ ”
 - ▶ etc.

What is induction?

- ▶ Induction is a convenient way to prove many (countably ∞) related mathematical statements at once
- ▶ What do we mean by “related mathematical statements”?
- ▶ Ex: let $P(n)$ be the statement “ $n^2 = 1 + 3 + 5 + \dots + (2n+1)$ ”
 - ▶ $P(1)$ is then “ $1^2 = 1$ ”
 - ▶ $P(2)$ is then “ $2^2 = 1 + 3$ ”
 - ▶ $P(3)$ is then “ $3^2 = 1 + 3 + 5$ ”
 - ▶ etc.
 - ▶ The statement “ $n^2 = 1 + 3 + 5 + \dots + (2n+1)$ ” is *indexed* by n

(Weak) Induction

- ▶ If we have a collection of statements $P(1)$, $P(2)$, ... then a (weak) inductive proof would look something like:

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$

(Weak) Induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a (weak) inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$



(Weak) Induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a (weak) inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for some integer $k \geq 1$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$



(Weak) Induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a (weak) inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for some integer $k \geq 1$
 - ▶ **Inductive Step:** prove $P(k+1)$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$



(Weak) Induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a (weak) inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for some integer $k \geq 1$
 - ▶ **Inductive Step:** prove $P(k+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$

(Weak) Induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a (weak) inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for some integer $k \geq 1$
 - ▶ **Inductive Step:** prove $P(k+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$

(Weak) Induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a (weak) inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for some integer $k \geq 1$
 - ▶ **Inductive Step:** prove $P(k+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$

(Weak) Induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a (weak) inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for some integer $k \geq 1$
 - ▶ **Inductive Step:** prove $P(k+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$

(Weak) Induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a (weak) inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for some integer $k \geq 1$
 - ▶ **Inductive Step:** prove $P(k+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(k) \Rightarrow P(k+1) \Rightarrow \dots$$

Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(n) \Rightarrow P(n+1) \Rightarrow \dots$$

Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(n) \Rightarrow P(n+1) \Rightarrow \dots$$



Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for all integers k where $1 \leq k \leq n$

$$\begin{array}{ccccccc} P(1) & \Rightarrow & P(2) & \Rightarrow & \dots & \Rightarrow & P(n) \Rightarrow P(n+1) \Rightarrow \dots \\ \checkmark & & \checkmark & & \checkmark & & \checkmark \end{array}$$

Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for all integers k where $1 \leq k \leq n$
 - ▶ **Inductive Step:** prove $P(n+1)$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(n) \Rightarrow P(n+1) \Rightarrow \dots$$



Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for all integers k where $1 \leq k \leq n$
 - ▶ **Inductive Step:** prove $P(n+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(n) \Rightarrow P(n+1) \Rightarrow \dots$$

Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for all integers k where $1 \leq k \leq n$
 - ▶ **Inductive Step:** prove $P(n+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(n) \Rightarrow P(n+1) \Rightarrow \dots$$

Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for all integers k where $1 \leq k \leq n$
 - ▶ **Inductive Step:** prove $P(n+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(n) \Rightarrow P(n+1) \Rightarrow \dots$$

Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for all integers k where $1 \leq k \leq n$
 - ▶ **Inductive Step:** prove $P(n+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(n) \Rightarrow P(n+1) \Rightarrow \dots$$

Strong induction

- ▶ If we have a collection of statements $P(1), P(2), \dots$ then a strong inductive proof would look something like:
 - ▶ **Base Case:** prove $P(1)$
 - ▶ **Inductive Hypothesis:** assume $P(k)$ is true for all integers k where $1 \leq k \leq n$
 - ▶ **Inductive Step:** prove $P(n+1)$
 - ▶ **Conclusion:** $P(n)$ is true for $n = 1, 2, 3, \dots$

$$P(1) \Rightarrow P(2) \Rightarrow \dots \Rightarrow P(n) \Rightarrow P(n+1) \Rightarrow \dots$$

Asymptotic Notation

uh oh, nΩbωdy knΘws the nOtatiOn

- There's a lot of symbols: o, O, ω, Ω, Θ, Ω₊, Ω₋, Ω_±

uh oh, nΩbωdy knΘws the nOtatiOn

- ▶ There's a lot of symbols: o, O, ω, Ω, Θ, Ω₊, Ω₋, Ω_±
- ▶ Why so many?

uh oh, nΩbωdy knΘws the nOtatiOn

- ▶ There's a lot of symbols: o , O , ω , Ω , Θ , Ω_+ , Ω_- , $\Omega_{\pm}$
- ▶ Why so many?
 - ▶ Mathematicians love to be precise!

uh oh, nobody knows the notation

- ▶ There's a lot of symbols: o , O , ω , Ω , Θ , Ω_+ , Ω_- , $\Omega_{\pm}$
- ▶ Why so many?
 - ▶ Mathematicians love to be precise!
 - ▶ ... but also lazy

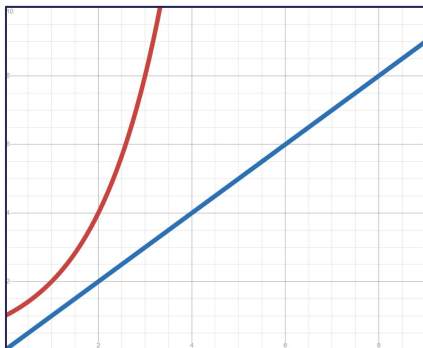
uh oh, nΩbωdy knΘws the nOtatiOn

- ▶ There's a lot of symbols: o , O , ω , Ω , Θ , Ω_+ , Ω_- , $\Omega_{\pm}$
- ▶ Why so many?
 - ▶ Mathematicians love to be precise!
 - ▶ ... but also lazy
- ▶ There's more than these, but you only need to worry about O , Ω , Θ

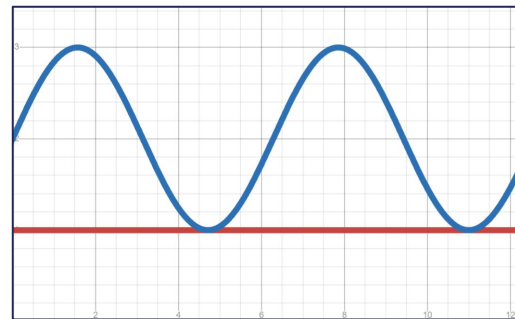
Big-O is an UPPER BOUND

- **Formal Definition:** we say $f(n) = O(g(n))$ if there exists $n_0 > 0$, $c > 0$ such that for all $n \geq n_0$ we have $f(n) \leq c * g(n)$
- **Limits:** If the limit of $f(n)/g(n)$ exists as $n \rightarrow \infty$, then $f(n) = O(g(n))$ if and only if the limit is finite

- $f(n) = n$
- $g(n) = 2^n$
- Choose $n_0 = 1$
- Choose $c = 1$
- $\Rightarrow f(n) = O(g(n))$



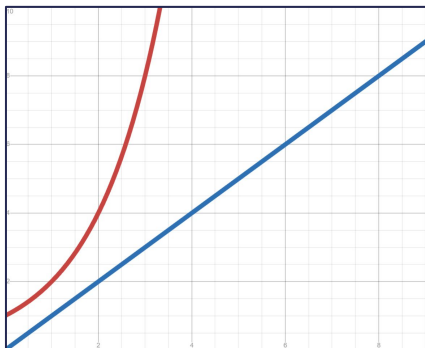
- $f(n) = 2 + \sin(n)$
- $g(n) = 1$
- Choose $n_0 = 1$
- Choose $c = 3$
- $\Rightarrow f(n) = O(g(n))$



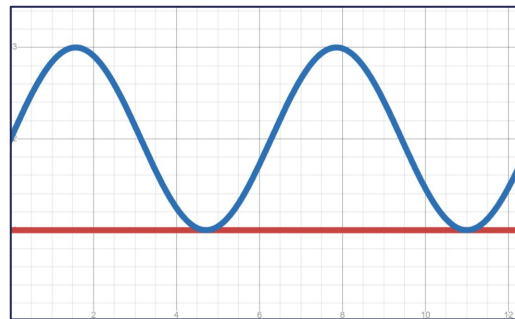
Big-Ω is an LOWER BOUND

- ▶ **Formal Definition:** we say $f(n) = \Omega(g(n))$ if there exists $n_0 > 0$, $c > 0$ such that for all $n \geq n_0$ we have $f(n) \geq c * g(n)$
- ▶ **Limits:** If the limit of $f(n)/g(n)$ exists as $n \rightarrow \infty$, then $f(n) = \Omega(g(n))$ if and only if the limit is nonzero
- ▶ **Helpful Hint:** $f(n) = \Omega(g(n)) \Leftrightarrow g(n) = O(f(n))$

- ▶ $f(n) = n$
- ▶ $g(n) = 2^n$
- ▶ Choose $n_0 = 1$
- ▶ Choose $c = 1$
- ▶ $\Rightarrow g(n) = \Omega(f(n))$



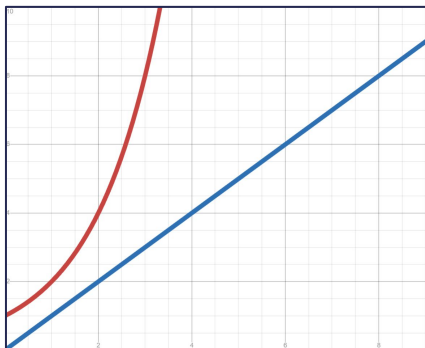
- ▶ $f(n) = 2 + \sin(n)$
- ▶ $g(n) = 1$
- ▶ Choose $n_0 = 1$
- ▶ Choose $c = 3$
- ▶ $\Rightarrow g(n) = \Omega(f(n))$



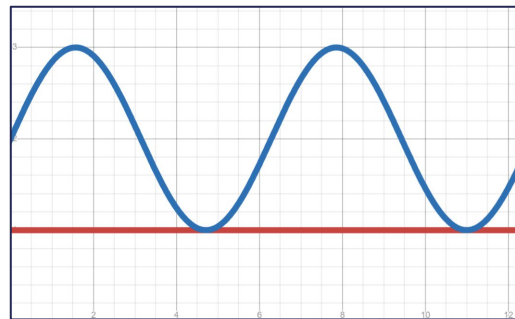
Big- Θ is an **UPPER** and **LOWER BOUND**

- **Formal Definition:** we say $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$
- **Limits:** If the limit of $f(n)/g(n)$ exists as $n \rightarrow \infty$, then $f(n) = \Theta(g(n))$ if and only if the limit is finite and nonzero

- $f(n) = n$
- $g(n) = 2^n$
- $f(n) = \Theta(g(n))$
is false



- $f(n) = 2 + \sin(n)$
- $g(n) = 1$
- $f(n) = \Theta(g(n))$
is true



The Potential Method

What is the potential method?

- ▶ The potential method helps us reason about:

What is the potential method?

- ▶ The potential method helps us reason about:
 - ▶ Whether or not a given algorithm will terminate

What is the potential method?

- ▶ The potential method helps us reason about:
 - ▶ Whether or not a given algorithm will terminate
 - ▶ How long it will take an algorithm to terminate

What is the potential method?

- ▶ The potential method helps us reason about:
 - ▶ Whether or not a given algorithm will terminate
 - ▶ How long it will take an algorithm to terminate
- ▶ **Formal Definition:** let S be a function mapping the state of an algorithm to $\mathbb{R}$. We call S a *potential function* if:

What is the potential method?

- ▶ The potential method helps us reason about:
 - ▶ Whether or not a given algorithm will terminate
 - ▶ How long it will take an algorithm to terminate
- ▶ **Formal Definition:** let S be a function mapping the state of an algorithm to $\mathbb{R}$. We call S a *potential function* if:
 - ▶ The output of S decreases by *at least* some fixed positive constant $C > 0$ after every well-defined “unit of work” (e.g., a loop iteration)

What is the potential method?

- ▶ The potential method helps us reason about:
 - ▶ Whether or not a given algorithm will terminate
 - ▶ How long it will take an algorithm to terminate
- ▶ **Formal Definition:** let S be a function mapping the state of an algorithm to $\mathbb{R}$. We call S a *potential function* if:
 - ▶ The output of S decreases by *at least* some fixed positive constant $C > 0$ after every well-defined “unit of work” (e.g., a loop iteration)
 - ▶ The output of S has some lower bound B such that the algorithm terminates whenever the output S is less than or equal to B

Practice Problems

Q1: Comparing asymptotic running times

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

Q1: Comparing asymptotic running times

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

- ▶ **READ CAREFULLY:** the input is fixed; it *does not* need to be a worst-case input

Q1: Comparing asymptotic running times

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

- ▶ **READ CAREFULLY:** the input is fixed; it *does not* need to be a worst-case input
- ▶ **REMEMBER:** big-O notation only cares about the input size *as it approaches infinity*

Q1: Comparing asymptotic running times

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

- ▶ **READ CAREFULLY:** the input is fixed; it *does not* need to be a worst-case input
- ▶ **REMEMBER:** big-O notation only cares about the input size *as it approaches infinity*
- ▶ **REMEMBER:** big-O is just an *upper bound*, that upper bound does not need to be “tight”

Q1: Comparing asymptotic running times SOLUTION

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

- ▶ **READ CAREFULLY:** the input is fixed; it *does not* need to be a worst-case input
 - ▶ **REMEMBER:** big-O notation only cares about the input size *as it approaches infinity*
 - ▶ **REMEMBER:** big-O is just an *upper bound*, that upper bound does not need to be “tight”
- | | | |
|---------------------|----------------|----------------------------|
| ▶ Running times: | Input size: 16 | ▶ Alg A will be: |
| ▶ Alg A: $n \log n$ | | ▶ Slower than Alg B |
| ▶ Alg B: n | | ▶ Faster than Alg C |
| ▶ Alg C: n^2 | | |

Attendance



Q2: Incorrect correctness

For each of the proposed “proofs” of the following (true!) statements, determine what is incorrect about the proof

Q2: Incorrect correctness

Claim: There is no even prime greater than 2.

Proof: 3 is a prime that is greater than 2 and it is not even, so there must be no even prime greater than 2.

Q2: Incorrect correctness

Claim: There is no even prime greater than 2.

Proof: 3 is a prime that is greater than 2 and it is not even, so there must be no even prime greater than 2.

This only proves that *not all* primes greater than 2 are not even.

Q2: Incorrect correctness

Claim: If n^2 is even, then n is even.

Proof: If n is even, then $n = 2k$ for some integer k . Therefore, $n^2 = 4k^2$
 $= 2 \cdot 2k^2$ is even

Q2: Incorrect correctness

Claim: If n^2 is even, then n is even.

Proof: If n is even, then $n = 2k$ for some integer k . Therefore, $n^2 = 4k^2$
 $= 2 \cdot 2k^2$ is even

This actually proves the *converse* of the claim.

Q2: Incorrect correctness

Claim: The sum of the first i positive odd integers equals the i^{th} square

Proof: $1 = 1^2$, and $1 + 3 = 4 = 2^2$, and $1 + 3 + 5 = 9 = 3^2$, and so on. The pattern will continue, so the claim is proven.

Q2: Incorrect correctness

Claim: The sum of the first i positive odd integers equals the i^{th} square

Proof: $1 = 1^2$, and $1 + 3 = 4 = 2^2$, and $1 + 3 + 5 = 9 = 3^2$, and so on. The pattern will continue, so the claim is proven.

Mathematicians are lazy, but we can't be *that* lazy.

We need to use a formal inductive argument to prove this claim.

Q2: Incorrect correctness

Claim: 37 is the largest prime smaller than 41.

Proof: 37 is less than 41, and is prime since its only divisors are 1 and 37.

Q2: Incorrect correctness

Claim: 37 is the largest prime smaller than 41.

Proof: 37 is less than 41, and is prime since its only divisors are 1 and 37.

We haven't shown that 38, 39, and 40 are not prime.

Q3: Geometric induction

Let $a, a*r, a*r^2, \dots$ be a geometric sequence.

Prove using induction that the sum of the first n terms is equal to

$$S_n = a(1 - r^n)/(1 - r).$$

Q3: Geometric induction

Base Case: We want to show $S_1 = a(1 - r^1)/(1 - r)$

Q3: Geometric induction

Base Case: We want to show $S_1 = a(1 - r^1)/(1 - r)$

Proof: $S_1 = a = a(1 - r)/(1 - r) = a(1 - r^1)/(1 - r)$

Q3: Geometric induction

Inductive Hypothesis: $S_k = a(1 - r^k)/(1 - r)$ for some $k \geq 1$

Q3: Geometric induction

Inductive Hypothesis: $S_k = a(1 - r^k)/(1 - r)$ for some $k \geq 1$

Inductive Step: We want to show $S_{k+1} = a(1 - r^{k+1})/(1 - r)$

Q3: Geometric induction

Inductive Hypothesis: $S_k = a(1 - r^k)/(1 - r)$ for some $k \geq 1$

Inductive Step: We want to show $S_{k+1} = a(1 - r^{k+1})/(1 - r)$

Proof: $S_{k+1} = a + a*r + a*r^2 + \dots + a*r^k$

Q3: Geometric induction

Inductive Hypothesis: $S_k = a(1 - r^k)/(1 - r)$ for some $k \geq 1$

Inductive Step: We want to show $S_{k+1} = a(1 - r^{k+1})/(1 - r)$

Proof: $S_{k+1} = a + a*r + a*r^2 + \dots + a*r^k = S_k + a*r^k$

Q3: Geometric induction

Inductive Hypothesis: $S_k = a(1 - r^k)/(1 - r)$ for some $k \geq 1$

Inductive Step: We want to show $S_{k+1} = a(1 - r^{k+1})/(1 - r)$

Proof:
$$S_{k+1} = a + a*r + a*r^2 + \dots + a*r^k = S_k + a*r^k$$
$$= a(1 - r^k)/(1 - r) + a*r^{k+1}$$

Q3: Geometric induction

Inductive Hypothesis: $S_k = a(1 - r^k)/(1 - r)$ for some $k \geq 1$

Inductive Step: We want to show $S_{k+1} = a(1 - r^{k+1})/(1 - r)$

Proof:
$$S_{k+1} = a + a*r + a*r^2 + \dots + a*r^k = S_k + a*r^k$$
$$= a(1 - r^k)/(1 - r) + a*r^{k+1} = a(1 - r^k)/(1 - r) + a(r^k - r^{k+1})/(1 - r)$$

Q3: Geometric induction

Inductive Hypothesis: $S_k = a(1 - r^k)/(1 - r)$ for some $k \geq 1$

Inductive Step: We want to show $S_{k+1} = a(1 - r^{k+1})/(1 - r)$

Proof:
$$\begin{aligned} S_{k+1} &= a + a^*r + a^*r^2 + \dots + a^*r^k = S_k + a^*r^k \\ &= a(1 - r^k)/(1 - r) + a^*r^{k+1} = a(1 - r^k)/(1 - r) + a(r^k - r^{k+1})/(1 - r) \\ &= a(1 - r^k + r^k - r^{k+1})/(1 - r) \end{aligned}$$

Q3: Geometric induction

Inductive Hypothesis: $S_k = a(1 - r^k)/(1 - r)$ for some $k \geq 1$

Inductive Step: We want to show $S_{k+1} = a(1 - r^{k+1})/(1 - r)$

Proof:
$$\begin{aligned} S_{k+1} &= a + a^*r + a^*r^2 + \dots + a^*r^k = S_k + a^*r^k \\ &= a(1 - r^k)/(1 - r) + a^*r^{k+1} = a(1 - r^k)/(1 - r) + a(r^k - r^{k+1})/(1 - r) \\ &= a(1 - r^k + r^k - r^{k+1})/(1 - r) = a(1 - r^{k+1})/(1 - r) \end{aligned}$$

Q4: This one's got some potential

Suppose *input* is a function that returns a user-specified positive integer. For each of the following problems, do the following tasks:

- ▶ Determine whether the following program halts for any sequence of (valid) inputs.
- ▶ If so, provide a proof using the potential method; if not, describe a sequence of inputs that causes the program to never halt

Q4: This one's got some potential

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

Q4: This one's got some potential

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:      $z \leftarrow \text{input}()$ 
5:     if  $z$  is even then
6:          $x \leftarrow x - 1$ 
7:          $y \leftarrow y + 1$ 
8:     else
9:          $y \leftarrow y - 1$ 
```

- Use the potential function $2x + y$.

Q4: This one's got some potential

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:      $z \leftarrow \text{input}()$ 
5:     if  $z$  is even then
6:          $x \leftarrow x - 1$ 
7:          $y \leftarrow y + 1$ 
8:     else
9:          $y \leftarrow y - 1$ 
```

- ▶ Use the potential function $2x + y$.
- ▶ If z is even, then
 - ▶ $2x + y \rightarrow 2(x - 1) + (y + 1) = 2x + y - 1$

Q4: This one's got some potential

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

- ▶ Use the potential function $2x + y$.
- ▶ If z is even, then
 - ▶ $2x + y \rightarrow 2(x - 1) + (y + 1) = 2x + y - 1$
- ▶ If z is odd, then
 - ▶ $2x + y \rightarrow 2x + (y - 1) = 2x + y - 1$

Q4: This one's got some potential

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

- ▶ Use the potential function $2x + y$.
- ▶ If z is even, then
 - ▶ $2x + y \rightarrow 2(x - 1) + (y + 1) = 2x + y - 1$
- ▶ If z is odd, then
 - ▶ $2x + y \rightarrow 2x + (y - 1) = 2x + y - 1$
- ▶ The potential decreases by at least 1 after each iteration of the while loop

Q4: This one's got some potential

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

- ▶ Use the potential function $2x + y$.
- ▶ If z is even, then
 - ▶ $2x + y \rightarrow 2(x - 1) + (y + 1) = 2x + y - 1$
- ▶ If z is odd, then
 - ▶ $2x + y \rightarrow 2x + (y - 1) = 2x + y - 1$
- ▶ The potential decreases by at least 1 after each iteration of the while loop
- ▶ We know once the potential is less than or equal to 0, we'll exit the while loop

Q4: This one's got some potential

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
10:     $x \leftarrow x + 1$  // This line differs from the program in (a)
```

Q4: This one's got some potential

► $x \leftarrow 2$

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
10:     $x \leftarrow x + 1$  // This line differs from the program in (a)
```

Q4: This one's got some potential

► $x \leftarrow 2$

► $y \leftarrow 2$

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
10:     $x \leftarrow x + 1$  // This line differs from the program in (a)
```

Q4: This one's got some potential

- ▶ $x \leftarrow 2$
 - ▶ $y \leftarrow 2$
 - ▶ On the i^{th} iteration of the while loop, $z \leftarrow i$
- ```
1: $x \leftarrow \text{input}()$
2: $y \leftarrow \text{input}()$
3: while $x > 0$ and $y > 0$ do
4: $z \leftarrow \text{input}()$
5: if z is even then
6: $x \leftarrow x - 1$
7: $y \leftarrow y + 1$
8: else
9: $y \leftarrow y - 1$
10: $x \leftarrow x + 1$ // This line differs from the program in (a)
```

## Q4: This one's got some potential

```
1: $x \leftarrow \text{input}()$
2: $y \leftarrow \text{input}()$
3: while $x > 0$ and $y > 0$ do
4: $z \leftarrow \text{input}()$
5: if z is even then
6: $x \leftarrow x - 1$
7: $y \leftarrow y + 1$
8: else
9: $y \leftarrow y - 1$
10: $x \leftarrow x + 1$ // This line differs from the program in (a)
```

- ▶  $x \leftarrow 2$
- ▶  $y \leftarrow 2$
- ▶ On the  $i^{\text{th}}$  iteration of the while loop,  $z \leftarrow i$
- ▶ If we represent the state as  $(x, y)$ , then the state evolves as follows:

## Q4: This one's got some potential

```
1: $x \leftarrow \text{input}()$
2: $y \leftarrow \text{input}()$
3: while $x > 0$ and $y > 0$ do
4: $z \leftarrow \text{input}()$
5: if z is even then
6: $x \leftarrow x - 1$
7: $y \leftarrow y + 1$
8: else
9: $y \leftarrow y - 1$
10: $x \leftarrow x + 1$ // This line differs from the program in (a)
```

- ▶  $x \leftarrow 2$
- ▶  $y \leftarrow 2$
- ▶ On the  $i^{\text{th}}$  iteration of the while loop,  $z \leftarrow i$
- ▶ If we represent the state as  $(x, y)$ , then the state evolves as follows:
  - ▶  $(2, 2) \rightarrow (3, 1) \rightarrow (2, 2) \rightarrow (3, 1) \rightarrow \dots$

## Q4: This one's got some potential

```
1: $x \leftarrow \text{input}()$
2: $y \leftarrow \text{input}()$
3: while $x > 0$ and $y > 0$ do
4: $z \leftarrow \text{input}()$
5: if z is even then
6: $x \leftarrow x - 1$
7: $y \leftarrow y + 1$
8: else
9: $y \leftarrow y - 1$
10: $x \leftarrow x + 1$ // This line differs from the program in (a)
```

- ▶  $x \leftarrow 2$
- ▶  $y \leftarrow 2$
- ▶ On the  $i^{\text{th}}$  iteration of the while loop,  $z \leftarrow i$
- ▶ If we represent the state as  $(x, y)$ , then the state evolves as follows:
  - ▶  $(2, 2) \rightarrow (3, 1) \rightarrow (2, 2) \rightarrow (3, 1) \rightarrow \dots$
- ▶ It never terminates, so we *know* that there doesn't exist a potential function proving it terminates

Any questions?